Typed Evidence Handoffs in a Coding-Agent Takeover: A Single-Task SWE-bench Verified Pilot

Bagdat Beimzhan

Exploratory empirical pilot | 27 October 2026

Keywords: coding agents; agent handoff; context management; software engineering; SWE-bench Verified

Abstract

A coding-agent handoff transfers repository state and a record of prior work to a successor, but one task run cannot establish whether a representation is generally better. We report a five-condition comparison on the SWE-bench Verified instance `pydata__xarray-6721`. A GPT-6 Sol predecessor inspected the issue, reproduced data access with a local lazy-array fixture, and was interrupted before its first source-edit action executed. The empty patch received an official `unresolved` grade. In a separate Docker replay of the benchmark tests, the target test failed with `UnexpectedDataAccess`; 1,405 pass-to-pass tests passed, 33 were skipped, one was xfailed, and five were xpassed among 1,445 collected.

A Claude Opus 5.5 successor ran once under each of five context conditions. All five patches received an official `resolved` grade. Successor input usage ranged from 44,126 tokens for `SUMMARY_2K` to 138,443 for `FULL_HISTORY`. `KEISHO_2K` used 47,170 input tokens and four model calls, 65.9% fewer input tokens than full history. The narrative and structured summaries used fewer input tokens than Keisho. Observed calls ranged from four to seven, and agent-run times from 32.7 to 86.8 seconds. With one task and one run per condition, these are descriptive observations. They do not establish a general accuracy or efficiency advantage for typed handoffs.

1. Introduction

When one coding agent takes over from another, the successor inherits a repository state and some account of earlier work. The account may contain executed observations, proposed changes, and unresolved questions. These have different evidential status. A proposed fix, for example, is not an applied or tested fix.

KC and Budathoki [1] studied this problem through handoffs at fixed points in coding-agent trajectories. Their evaluation compared repository-only context, full traces, summaries, and structured notes across multiple source tasks. Context-bearing handoffs reduced measured takeover effort relative to repository-only context, while solve-rate effects varied by model. Their work motivates evaluating continuation costs alongside task resolution.

This pilot asks a narrower question: on one unresolved SWE-bench Verified checkpoint, what resolution and resource-use observations result from five handoff conditions? The conditions are `REPOSITORY_ONLY`, `FULL_HISTORY`, `SUMMARY_2K`, `STRUCTURED_NOTES_2K`, and `KEISHO_2K`. The study does not test a broad claim that one condition is superior. It reports one completed continuation per condition on one task and makes the checkpoint and handoff contents explicit.

2. Related Work

SWE-bench evaluates whether a model-generated patch resolves a real GitHub issue under the benchmark's tests [2]. The present study uses one SWE-bench Verified instance and the official evaluator. It is a single-task takeover experiment, not a leaderboard result.

The closest comparison is Handoff Debt [1], which freezes repository state and evaluates successors under several views of a predecessor's work. Our design follows that takeover framing. It adds a condition assembled by Keisho from recorded events, but this pilot is not a reproduction of the Handoff Debt implementation or its multi-task evaluation. The local structured-notes baseline also adapts, rather than exactly reproduces, the published system.

ESAA-Conversational projects event-sourced conversation records into shared handoff and state files for heterogeneous coding agents [3]. MemTrace stores coding histories as provenance-aware traces and checks historical evidence against repository state [4]. Scroll keeps an event log and persistent execution environment for programmatic context management [5]. Each uses a different mechanism and evaluation setting. None was included as a baseline, so this pilot does not compare Keisho with them.

3. Method

3.1 Task and study unit

The task was `pydata__xarray-6721` from SWE-bench Verified. The repository was xarray at base commit `cc183652bf6e1273e985e1c4b3cba79c896c1193`. The issue concerns access to `Dataset.chunks` forcing data access for a lazily backed variable. In `get_chunksizes()`, the check `hasattr(v.data, "chunks")` can access `Variable.data` before determining whether chunk metadata is available.

The predecessor's local reproduction used a custom `LazilyIndexedArray` whose `__getitem__` raised `RuntimeError("loaded")`. Accessing `Dataset.chunks` triggered that exception. This was a local fixture, not a read from a live Zarr store.

The independent sampling unit for this report is the task. We executed one predecessor trajectory and one successor trajectory per condition. The five branches share a checkpoint and are not five independent task samples.

3.2 Checkpoint and successor conditions

The predecessor was `openai/gpt-6-sol-20260922`. It made six model calls and six tool calls. After reproducing the access, it issued a shell action that would have modified `xarray/core/common.py` and `xarray/core/variable.py`. The interceptor paused the action before execution. No source edit from that action reached the workspace.

The successor was `anthropic/claude-opus-5.5-20260921`. Each branch received the same issue, base image, tool interface, takeover instructions, and continuation budgets in a fresh workspace. The condition changed the predecessor context:

1. `REPOSITORY_ONLY` received no predecessor-specific information.
2. `FULL_HISTORY` received the serialized predecessor conversation, including tool actions and outputs.
3. `SUMMARY_2K` received a narrative summary generated by `openai/gpt-6-luna-20260922`.
4. `STRUCTURED_NOTES_2K` received a seven-field note generated by the same model, plus deterministic continuation facts.
5. `KEISHO_2K` received a locally compiled set of typed items from recorded events. The bundle distinguished recorded observations from hypotheses and retained unknowns. It did not contain the unexecuted source-edit action.

The three compact conditions had a 2,048-token handoff limit. This limit applied to the handoff payload, not the full model request. Keisho compilation used no additional model call. The two generated note conditions used one generation call each. The local structured-notes condition was adapted from the published seven-field approach; it did not use the authors' complete runtime or processing pipeline.

3.3 Procedure and measures

The five successor branches ran sequentially in this fixed order: `FULL_HISTORY`, `SUMMARY_2K`, `REPOSITORY_ONLY`, `KEISHO_2K`, then `STRUCTURED_NOTES_2K`. Each branch started from the same frozen checkpoint in an independent container. No branch saw another branch's output. Anthropic's route did not support the requested seed parameter, so the successor calls had no provider-side seed pinning.

We recorded the official SWE-bench status, provider-reported input and output tokens, model and tool call counts, and runner-reported elapsed time for each successor. Elapsed time covers the successor run and its tool interactions; it is not a measure of model reasoning time alone. The branches ran one after another, so the time observations are not randomized comparisons.

Input-token totals in Table 1 cover successor calls only. They exclude the predecessor and note-generation requests. The reported total provider charge covers all stages of the completed run. The official evaluator graded each exported patch in Docker; an agent's own completion message did not determine the grade.

4. Results

4.1 Checkpoint verification

The official evaluator classified the predecessor's empty patch as `unresolved` with `empty_patch: true` and exit code 0. Because this empty-patch path did not run the target tests, we separately executed the benchmark test patch in the pinned Docker image. Pytest collected 1,445 tests. The target test, `test_chunks_does_not_load_data`, failed with `xarray.tests.UnexpectedDataAccess: Tried accessing data`. Of the baseline tests, 1,405 passed, 33 were skipped, one was xfailed, and five were xpassed. This replay confirms that the checkpoint reproduced the benchmark's target failure; it does not test a live remote Zarr dataset.

4.2 Successor outcomes

Table 1. Official grade and successor-run resource observations for each condition in the completed exploratory comparison.

Condition	Official grade	Model calls	Tool calls	Input tokens	Output tokens	Elapsed time (s)
`FULL_HISTORY`	resolved	4	3	138,443	1,585	32.7
`SUMMARY_2K`	resolved	5	4	44,126	1,931	86.8
`REPOSITORY_ONLY`	resolved	7	6	54,526	2,315	53.8
`KEISHO_2K`	resolved	4	3	47,170	1,493	34.5
`STRUCTURED_NOTES_2K`	resolved	5	4	45,429	1,650	86.5

All five branches received an official `resolved` grade. Each patch changed `xarray/core/common.py`; four also changed `xarray/core/variable.py`. The grade was identical across conditions, although patch contents and execution paths varied.

4.3 Information transferred

The context payloads did not contain equivalent information in different formats. `FULL_HISTORY` included the predecessor's unexecuted patch action. The narrative summary and structured notes also described the proposed changes. `KEISHO_2K` omitted the unexecuted write action and retained the observed reproduction and a hypothesis about lazy data access. `REPOSITORY_ONLY` contained no predecessor findings.

This distinction matters when interpreting call counts. The experiment compared complete handoff conditions, including their content-selection rules. It did not isolate the effect of typed labels while holding the substantive information constant.

4.4 Resource observations

SUMMARY_2K used the fewest successor input tokens (44,126), followed by STRUCTURED_NOTES_2K (45,429), KEISHO_2K (47,170), REPOSITORY_ONLY (54,526), and FULL_HISTORY (138,443). Relative to full history, the observed reductions were 68.1% for the summary, 67.2% for structured notes, 65.9% for Keisho, and 60.6% for repository-only.

FULL_HISTORY and KEISHO_2K each used four model calls and three tool calls. The two generated-note conditions used five model calls and four tool calls; the repository-only condition used seven and six. In the summary branches, the additional call included code inspection before the edit. This trace does not establish why the model took that step.

Runner-reported elapsed time ranged from 32.7 seconds for FULL_HISTORY to 86.8 seconds for SUMMARY_2K. KEISHO_2K took 34.5 seconds. These durations include model-service and tool latency and were observed in a fixed sequential order. They do not establish a latency effect caused by handoff representation.

The completed run used 33 settled provider calls: six for the predecessor, two for note generation, and 25 across the successors. Actual provider charges were USD 1.534741. Conservative accounted spend was USD 1.590336, with no unreconciled reservation for this run. These totals include note-generation costs; Table 1's token counts do not.

5. Discussion

The direct result is that all five successor conditions resolved this one issue under the tested model pair and harness. Since the resolution outcome tied at 5/5, the run cannot rank the conditions by task success. The repository-only successor also solved the issue, so predecessor context was not necessary for resolution in this instance.

Keisho used fewer input tokens than full history and the same number of model and tool calls. It did not use the fewest input tokens: both generated summaries were shorter at the successor stage. The summary and structured-note branches took one additional model and tool call in this run. The trace shows an inspection step before their edits, but it does not show that typed provenance caused Keisho to skip that step. Patch-proposal content, one-run model variability, fixed execution order, and provider latency remain alternative explanations.

The completed comparison is useful as a protocol and measurement record. It shows that an unresolved checkpoint can be verified independently of the benchmark's empty-patch shortcut, and it records how five handoff conditions behaved on the same task. It does not establish that Keisho improves task completion, prevents hallucinations, or reduces total system cost on other tasks.

6. Limitations

First, the evaluation covers one task and one run per condition. There is no estimate of between-task or between-run variability, no confidence interval, and no statistical test. All branches resolved, leaving no outcome variation to analyze.

Second, the experiment used one predecessor model, one successor model, and one note-generation model. The successor route did not support provider-side seed pinning. Results may change with model, provider, routing, or sampling.

Third, handoff content differed as well as format. The full history and generated notes exposed the proposed patch; the Keisho bundle omitted the unexecuted write action. The comparison cannot identify a causal effect of typed representation independent of what each payload preserved.

Fourth, STRUCTURED_NOTES_2K was a local adaptation of the published structured-note approach. Its source packet and processing behavior differed from the authors' released implementation. The result should not be read as an author-faithful baseline comparison.

Fifth, the Keisho bundle retained unknown evidence scopes. Although the broader system supports file-scope checks, this run does not show that a complete file-hash revalidation mechanism improved successor behavior. A successful SWE-bench grade also establishes only that the exported patch passed the benchmark's grading tests; it does not establish behavior on every lazy storage backend or in deployment.

Finally, the completed run followed four earlier attempts that exposed protocol and instrumentation failures. The successful run was exploratory and followed protocol repairs. The earlier attempts are disclosed in Appendix A and are not pooled with the five successor outcomes.

7. Conclusion

On one unresolved xarray checkpoint, all five successor handoff conditions received an official resolved grade. `KEISHO_2K` used 47,170 successor input tokens and four model calls, compared with 138,443 tokens and four calls for `FULL_HISTORY`. The narrative and structured summaries used fewer input tokens than Keisho and one additional call. These single-run observations do not establish a general advantage for any condition.

A follow-up should vary the handoff representation while controlling the substantive information transferred, then repeat the comparison across independently sampled tasks and model runs. One specific question is whether provenance and uncertainty labels change how successors reuse or revalidate evidence when no exact patch proposal is handed over.

Data and Code Availability

The completed run records remain in a local archive identified in Appendix B. The checkpoint verification log and structured result are under `analysis/checkpoint_verification_log.txt` and `analysis/checkpoint_verification_result.json`. The run archive is not publicly available, and no public archive or DOI is provided in this manuscript version.

References

[1] D. KC and A. Budathoki, “Handoff Debt: The Rediscovery Cost When Coding Agents Take Over Interrupted Tasks,” arXiv:2606.02875, version 2, 2026. https://arxiv.org/abs/2606.02875

[2] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” in *Proceedings of the Twelfth International Conference on Learning Representations*, 2024. arXiv:2310.06770. https://arxiv.org/abs/2310.06770

[3] E. Brito dos Santos Filho, “ESAA-Conversational: An Event-Sourced Memory Layer for Continuity, Handoff, and Curation Across Heterogeneous LLM Coding Agents,” arXiv:2606.23752, 2026. https://arxiv.org/abs/2606.23752

[4] H. Xu, L. Zhou, Z. Jin, X. Zhang, B. Tang, Z. Li, X. Zhou, and J. Zhang, “MemTrace: State-Consistent Memory for Long-Horizon Coding Agents,” arXiv:2610.04838, version 3, 2026. https://arxiv.org/abs/2610.04838

[5] Y. Lin, E. Ang, E. Zhu, B. Ding, and J. Zhou, “Context as an Environment: Programmatic Context Management for Long-Horizon Agents,” arXiv:2608.21690, 2026. https://arxiv.org/abs/2608.21690

Appendix A. Protocol Development Attempts

Four protocol-development attempts preceded the completed comparison. They are not additional observations for the five-condition results. Their charges are reported separately and are not pooled with Table 1.

Table A1. Provider charges and reservations across the five recorded attempts. Amounts are rounded to six decimal places.

Attempt	Provider requests	Outcome	Billed (USD)	Accounted (USD)	Unreconciled (USD)
001	8	No qualifying checkpoint; successor branches unstarted	0.069326	0.148786	0.000000
002	13	Predecessor finished the repair before a checkpoint; successors unstarted	0.065966	0.249584	0.000000
003	10	Unresolved checkpoint; successor request rejected for unsupported seed parameter	0.036516	0.105225	0.379008
004	14	Unresolved checkpoint; successor responses stopped by local token-bound checks before tool actions	0.344762	0.414938	0.000000
005	33	Completed five-condition comparison	1.534741	1.590336	0.000000

Attempt 001 failed to reach the stopping rule after the reproduction script caught its exception and exited with code 0. The predecessor then edited source files; a later response exceeded the output limit, and no successor branch started. In Attempt 002, the predecessor completed and tested a repair before the checkpoint rule fired, so all successor branches remained unstarted.

Attempt 003 reached an empty, officially unresolved checkpoint. The provider rejected the first successor request because the route did not accept the requested seed parameter. The ledger retains USD 0.379008 as unreconciled. Attempt 004 reached the same checkpoint, but each successor response exceeded the local input-token reservation bound before any tool action. In that run, one request was reported as 5,990 tokens by the OpenAI tokenizer while the provider reported 9,540 input tokens. Attempt 005 increased the reservation multiplier and completed all five branches.

Across the five recorded attempts, raw ledger values sum to USD 2.051310325 in actual charges and USD 2.508869925 in conservative accounted spend. The USD 0.379008 reservation from Attempt 003 remains unreconciled. Totals are calculated from unrounded ledger values; row displays are rounded to six decimals. A separate earlier v2.6 pilot on `sympy__sympy-20590` used a checkpoint whose fix was already applied. It measured solution preservation and is not combined with this unresolved-checkpoint comparison.

Appendix B. Run Specification

- Run: `swebench-v2.7-five-way-once-005`.
- Task: `pydata__xarray-6721`, SWE-bench Verified.
- Predecessor model: `openai/gpt-6-sol-20260922`.
- Note-generation model: `openai/gpt-6-luna-20260922`.
- Successor model: `anthropic/claude-opus-5.5-20260921`.
- Dataset revision: `c104f840cc67f8b6eec6f759ebc8b2693d585d4a`.
- Task-repository recipe commit: `3d07b464b7b311a0cbfb5ed5b2d8a3b96f84a33d`.
- Base commit: `cc183652bf6e1273e985e1c4b3cba79c896c1193`.
- Evaluation image: `swebench/sweb.eval.x86_64.pydata_1776_xarray-6721@sha256:58c7f4ada7f3cbcfcde50e0b55157701100cf694faa87978c672e368c0531898`.
- Successor order: `FULL_HISTORY`, `SUMMARY_2K`, `REPOSITORY_ONLY`, `KEISHO_2K`, `STRUCTURED_NOTES_2K`.
- Configuration SHA-256 bound to the run: `09f93905fbbfb3abf25092b19657385ae644cfbf78714e5a0a36eb56b735b6ff`.
- Plan SHA-256 bound to the run: `cc84d41d847ebcf3b3b1ed93b9dafe4ef0311f0315ffd4455cef09b56b84bcfc`.
- Primary result and cost records: `runs/swebench-v2.7-five-way-once-005/result.json` and `runs/swebench-v2.7-five-way-once-005/spending.json`.
- Checkpoint replay: `analysis/checkpoint_verification_command.sh`, `analysis/checkpoint_verification_log.txt`, and `analysis/checkpoint_verification_result.json`.